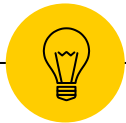




PingFin – students manual

April 2026

General outline

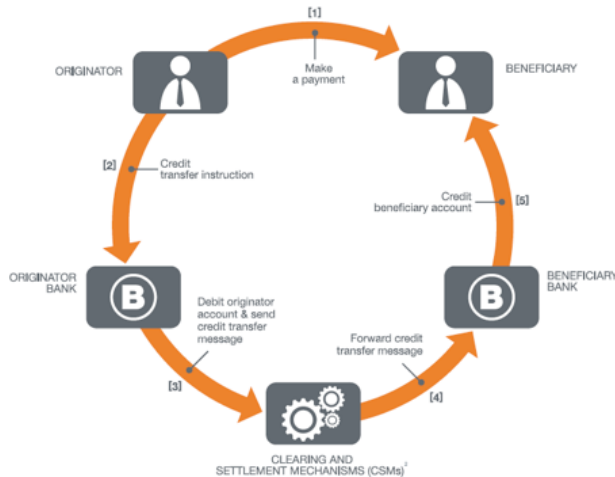




SEPA

This workshop aims at implementing a simplified version of a payments system within the Single Euro Payments Area (**SEPA**). Student teams play the role of either a *standard bank* sending/accepting payments (**OB**: originating bank; **BB**: beneficiary bank), or the *clearing bank* (**CB**), acting as a bank in the middle mediating payment orders.

SEPA CREDIT TRANSFER SCHEME MODEL



Every team needs to find an **efficient**, **secure** and **fail-safe** way to process and send payment orders to other banks via a clearing house and get a successful reply message back – like a "ping" used for testing connectivity in TCP/IP computer networks. Every bank needs to keep track of failed and successful **transactions** and other **events** and provide a basic **GUI**.

Planning and **communication** (internally and externally) is an additional challenge for all participants.

The use of tools like **chatGPT/Claude/...**, mock **data generators** etc.. is actually encouraged, but everyone should **understand** all generated code.



Bank Team – Responsibilities

Each team will build an **application** (regular bank or clearing bank):

- study the SEPA general **messaging scheme**
- build a **database** containing tables for logs, payment orders (PO_IN, PO_OUT, ACK_IN...), transactions and accounts (regular banks), banks (clearing banks) etc...
- build an **API** with all necessary external and internal endpoints
- build a **GUI** for the end user on top of the API (no direct database access is allowed!)
- perform a series of **tests** by creating a group of PO's and running them through the system (OB – CB – BB)
- handle **exceptions** (API request or response errors, PO format or data errors...)
- handle **security** (API & GUI authentication and authorization)

Each bank team includes at least one student from the Cybersecurity and/or System Administrator track. Select one of them as the single point of contact with the network team, responsible for coordinating communication and leading the Docker setup within the team.



Network Team – Responsibilities

The **network team** has the following responsibilities:

- Set up and manage the local wired network infrastructure (switch, cabling, connectivity)
- Install, configure, and maintain the central server
- Ensure all bank applications are reachable over the network
- Collect required technical information from bank teams (ports, endpoints, configs, etc.)
- Support bank teams in Dockerizing their applications
- Deploy and manage Docker containers/images on the central server
- Monitor availability and connectivity of all deployed services
- Troubleshoot network, server, and deployment issues
- Ensure proper documentation of configurations and setup
- Maintain security best practices (basic firewalling, access control, network isolation if needed)
- Handle all hardware logistics (pickup, setup, and return of server, switch, cables – HER 3-5306)
- Ensure responsible and respectful use of equipment
- handle **security** (API & GUI authentication and authorization)



Network Team – Responsibilities

Additional responsibilities (for faster teams):

- Implement basic monitoring/dashboard (e.g. uptime, service status)
- Set up reverse proxy / routing (e.g. single entry point for all APIs)
- Configure automated deployment pipelines (CI/CD basics)
- Add logging and error tracking for deployed services
- Perform basic load testing or stress testing
- Improve network security setup (segmentation, HTTPS, certificates)



Process

Teams will also need to work on **organization, communication & planning**:

- define and assign **roles/responsibilities** to individual team members
- set up and configure a **repository**
- set up a **project management** tool (Kanban / Trello / Jira...) and log tasks (who what when)
- create an **API documentation**
- write a **technical document** with daily reports
- have daily **stand-up meetings** within the team
- do an **intermediary presentations** to other teams and the coach
- do a **final presentation** to other teams and the coach with a reflection on the process and product during this workshop week

Note: the process is at least as important as, if not more important than the end product!



Timeline

	activities	deliverables
day 1, morning "analysis & planning"	▪ coach explains the workshop (idea, scope, requirements) ▪ understanding the general SEPA messaging scheme and discussing strategies to recover from exceptions ▪ project setup: repository, management tools, tasks...	▪ report document in Word format with front page, introduction, visual representation of basic messaging use cases... ▪ repository ▪ project management setup (Kanban or Trello board, ...)
day 1, afternoon "design"	▪ short presentation of morning work ▪ Bruno / Postman simulation ▪ application design: technologies, schematic structure ▪ design and develop database ▪ populate with at least 20 accounts with 5000 euros each	▪ short presentation (Powerpoint or similar) ▪ Bruno / Postman screenshots ▪ SQL file with scheme + data ▪ application design (add to Word report document)
day 2 "implementation"	▪ design API ▪ implement + test some typical API endpoints ▪ generate PO's (min. 10) and test own API endpoints ▪ start building GUI	▪ API documentation ▪ mockups of the GUI
day 3 "testing & improvements"	▪ finalize API implementation ▪ put API online; test each other's API ▪ run tests, improve code ▪ finalize GUI ▪ implement validation, error & exception handling	▪ test results and self-reflection
day 4 "presentation"	▪ final presentation and retrospective by each team	▪ final presentation (Powerpoint or similar)



Practical information

The workshop will take place from **Monday 27th to Thursday 30th** of April, ending Thursday afternoon with final presentations

It takes place **on campus**, every day from 8h30 – 12h30 and 13h30 – 17h30 (*short* intermediary breaks are allowed)

Each day each team will start with a **stand-up meeting**

There are +/- **100 students**, 4 per team and **11 coaches**:

- Anissa Faik
- Dries Blanchaert
- Frank Salliau
- Frank Serneels
- Hans Vandenbogaerde
- Jens Baetens
- Polina Kozlova
- Rogier van der Linde
- Sam Van Buggenhout
- Steven Ophalvens
- Turgut Kursun



Evaluation

The final mark (on 20 points) consists of:

- 30% **assessment**
- 30% **product**
- 20% **process**
- 20% **documentation & presentation**

The marks of individual team members may differ according to their contribution (factor peer-assessment).

Please note that **attendance is mandatory** from beginning till end:

- *for each half day a student comes in **too late** or leaves **too early** without prior notice and accepted reason, **-1 point** is deducted*
- *for each half day a student is **absent** without prior notice and accepted reason, **-2 point** are deducted*
- *a student who is absent **more than one day**, cannot pass and will get a NA – even with doctor's notes!*



Evaluation details (Bank)

PRODUCT (10 pt)

- analysis and planning (day 1 morning - 2pt)
- design (day 1 afternoon - 2pt)
- implementation (day 2 - 2pt)
- testing & debugging (day 3 - 2pt)
- validation and exception handling (day 3 - 2pt)

PROCESS (10 pt)

Team member will **peer evaluate** each other (a public form will be made available). The coach awards an overall grade, taking into account the evaluation results.

PRESENTATION (2.5 pt)

- presentation day 1 (1pt)
- final presentation (1.5pt)

DOCUMENTATION (2.5 pt)

- process description (used tools, repo setup... 1pt)
- product description (mockups, application design... 1pt)
- self reflection (0.5pt)



Evaluation details (Network)

PRODUCT (10 pt)

- Analysis & planning (2 pt): requirements, planning, coordination
- Design (2 pt): network architecture, deployment strategy
- Implementation (2 pt): server & Docker setup, dummy containers
- Testing & debugging (2 pt): deployment of real apps, connectivity fixes
- Validation & exception handling (2 pt): robustness, handling failures

PROCESS (10 pt)

Team member will **peer evaluate** each other (a public form will be made available). The coach awards an overall grade, taking into account the evaluation results.

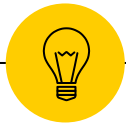
PRESENTATION (2.5 pt)

- Day 1 (1 pt): setup & plan
- Final (1.5 pt): architecture, deployment, demo

DOCUMENTATION (2.5 pt)

- Process (1 pt): tools, collaboration
- Product (1 pt): diagram, deployment, endpoints
- Self reflection (0.5 pt): lessons learned

Application requirements





Business rules

Business requirements:

- each step all transactions must be **datetimed** by the OB, CB and BB
- all PO's should contain a **message** with at least the **name of the test run**
- every bank should keep **track of all outstanding payments** for which an acknowledgement is missing
- every member bank should **keep track of failed transactions**
- the maximum value of a payment is **500 euros**
- every outstanding payment order should receive an acknowledgement **within 1 hour**
- an account balance can **not go below zero**
- every bank should keep a **log of all events** incoming and outgoing messages, exceptions etc...

*Note: we will use the term **datetime** instead of **timestamp** (which in SQL Server is not a time, but a versioning field)*



Business rules

Business requirements (cont'd):

- each amount should have no more than **two digits after the comma**
- each datetime should have the format **YYYY-MM-DD HH:MM:SS**
- each bank id should be a **valid BIC** (no spaces)
- each account id should be a **valid IBAN** code (no spaces)
- each payment order is assigned a **unique id**, prefixed with the BIC_ of the OB, e.g. GKCCBEBB_21xa-39-95, with a maximum length of 50 characters



Payment Order (PO) message format

po_id	ID generated by OB
po_amount	PO amount
po_message	PO message (payment purpose)
po_datetime	date and time PO created
ob_id	BIC of OB
oa_id	IBAN of OA
ob_code	code generated by OB
ob_datetime	date and time of most recent OB action
cb_code	code generated by CB
cb_datetime	date and time of most recent CB action
bb_id	BIC of BB
ba_id	IBAN of BA
bb_code	code generated by BB
bb_datetime	date and time of most recent BB action

List of abbreviations used in this manual:

OB: originator bank (BIC)

CB: clearing bank (BIC)

BB: beneficiary bank (BIC)

TX: transaction

PO: payment order

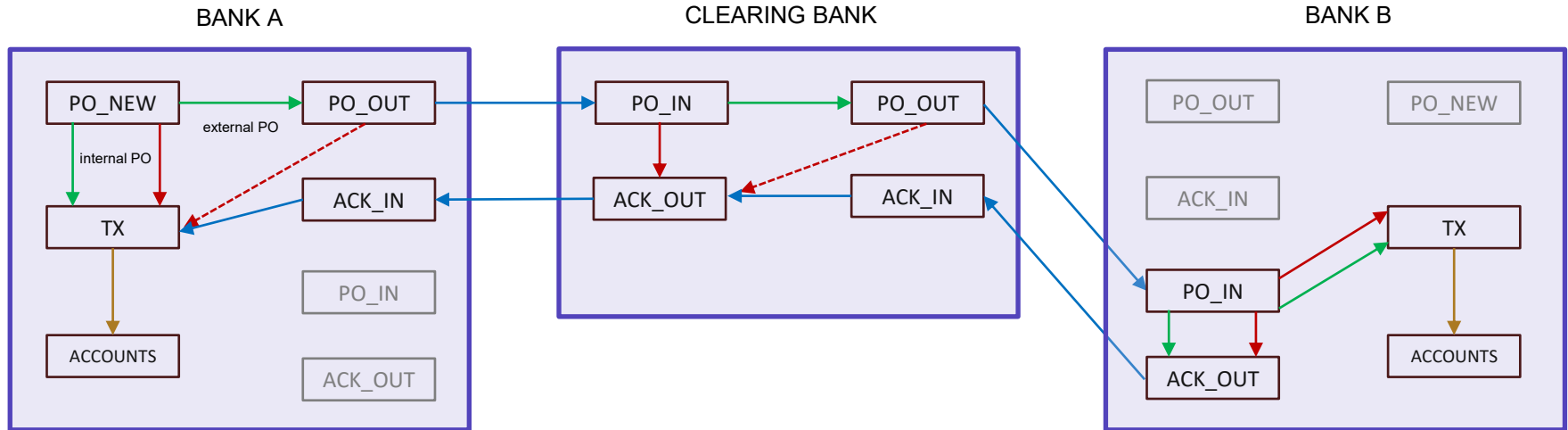
OA: originator account (IBAN)

BA: beneficiary account (IBAN)

- at the start of a payment order, a PO message is created
- this message runs through the whole SEPA process; each step may modify one or more fields of the PO message
- at the end of the process, the PO message is killed
- results will be stored in transaction and log tables



General messaging scheme





Database – BANK, ACCOUNTS, LOGS AND TRANSACTIONS

LOG, CB.LOG		
PK	id	log id
	datetime	log datetime
	message	log message
	type	log type (e.g. general, ack_in...)
N	po_id	PO ID generated by OB
N	po_amount	PO amount
N	po_message	PO message
N	po_datetime	date and time PO created
N	ob_id	BIC of OB
N	oa_id	IBAN of OA
N	ob_code	code generated by OB
N	ob_datetime	date and time of most recent OB action
N	cb_code	code generated by CB
N	cb_datetime	date and time of most recent CB action
N	bb_id	BIC of BB
N	ba_id	IBAN of BA
N	bb_code	code generated by BB
N	bb_datetime	date and time of most recent BB action

TRANSACTIONS		
PK	id	transaction id
	amount	amount (BA: positive, OA: negative)
	datetime	transaction datetime
	po_id	payment order id
	account_id	account IBAN
	isvalid	transaction successful or failed
	iscomplete	transaction complete

ACCOUNTS		
PK	id	IBAN
	balance	current balance

CB.BANKS		
PK	id	BIC of the bank
	name	bank name
	description	team members etc...
	token	bearer token

- BANKS table is only for clearing banks
- ACCOUNTS and TRANSACTIONS tables are only for regular banks
- LOG table is for both



Database – PO AND ACK TABLES

			PO_NEW	PO_OUT CB.PO_IN	PO_IN, ACK_IN CB.PO_OUT, CB.ACK_OUT,	ACK_OUT, CB.ACK_IN
PK	po_id	PO ID generated by OB				
	po_amount	PO amount				
	po_message	PO message				
	po_datetime	date and time PO created				
	ob_id	BIC of OB				
	oa_id	IBAN of OA				
	ob_code	code generated by OB	N			
	ob_datetime	date and time of most recent OB action	N			
	cb_code	code generated by CB	N	N		
	cb_datetime	date and time of most recent CB action	N	N		
	bb_id	BIC of BB				
	ba_id	IBAN of BA				
	bb_code	code generated by BB	N	N	N	
	bb_datetime	date and time of most recent BB action	N	N	N	

➤ *N: nullable (may be empty)*



API – clearing bank endpoints

These public clearing bank endpoints should be implemented by clearing banks:

URL	TYPE	BODY (JSON)	DESCRIPTION / RETURN (JSON)
/api/help/	GET	-	returns overview of API methods and usage (or a link or redirect to an external documentation)
/api/info/	GET	-	returns team info: members, bank BIC and name ...
/api/banks/	GET	-	returns a list of all banks
/api/po_in/	POST	list of PO's from OB	adds PO's to CB_PO_IN
/api/po_out/	GET	-	returns a list of PO's
/api/ack_in/	POST	list of PO's from BB	adds PO's to CB_ACK_IN
/api/ack_out/	GET	-	returns a list of PO's



API – regular bank endpoints

These public regular bank endpoints should be implemented by regular banks:

URL	TYPE	BODY (JSON)	DESCRIPTION / RETURN (JSON)
/api/help/	GET	-	returns overview of API methods and usage (or a link to an external documentation)
/api/info/	GET	-	returns team info: members, bank BIC and name ...
/api/accounts/	GET	-	returns a list of all accounts

Teams will also need extra endpoints for internal use; some suggestions (but feel free to design your own):

URL	TYPE	BODY (JSON)	DESCRIPTION / RETURN (JSON)
/api/po_new_generate/	GET	-	a list of random PO's
/api/po_new_add/	POST	list of PO's	adds new PO's to PO_NEW
/api/po_new_process/	GET	-	processes all PO's in PO_NEW
...	...	...	...



API – response format

Every response should contain at least these fields:

```
const response = {  
  ok: ... // true or false (success/fail)  
  status: ... // HTTP status code, e.g. 200 = ok, 404 = not found, 500 = server error...  
  code: ... // message or error code; null/undefined if none  
  message: ... // detailed message from API; null/undefined if none  
  data: ... // array of PO objects or null/undefined if none  
};
```



API – request format

Schematic POST request:

```
const listToSend = [{ ...P01... }, { ...P02... }, { ...P03... }, ...]; // a list of P0, if needed
const requestOptions = {
  headers: { 'Authorization': 'Bearer ghp_fIWhq1DrL...' }, // bearer token provided by CB
  method: 'POST',
  headers: myHeaders,
  body: JSON.stringify({ data: listToSend }) // only if needed
};

//... perform request
```

Schematic GET request:

```
const requestOptions = {
  headers: { 'Authorization': 'Bearer ghp_fIWhq1DrL...' }, // bearer token provided by CB
  method: 'GET',
  headers: myHeaders
};

//... perform request
```



Message codes

To allow logging and handling of messages and errors consistently across banks, we'll use message codes for the fields `ob_code`, `cb_code` and `bb_code`. Some examples (but feel free to create your own):

code	type	DESCRIPTION / RETURN (JSON)
2000		general OK code
4001	CB error	this is an internal transaction and should not be sent to the CB
4002	CB error	the transaction amount exceeds the 500 euros limit
4003	CB error	the transaction amount cannot be negative
4004	CB error	bb_id does not exist in the CB system
4005	CB error	a PO with this po_id has already been received by the Clearing Bank (pending)

See documentation: <https://stevenop.be/pingfin/api/v2/errorcodes>

Day by day (bank team)





Day 1 – morning: analysis & planning

Task A: Study and discuss the **general messaging scheme**. How do the fields of the PO change in each step? Try to cover all messaging cases and find a way to visualize them.

use case 1: OB validation fails (e.g. unknown OA account)

use case 2: internal payment, OB validation passes

use case 3: external payment, OB validation passes, CB validation fails (e.g. unknown BB bank)

use case 4: OB & CB validation pass, BB validation fails (e.g. unknown BA account)

use case 5: OB, CB & BB validation all pass

Also discuss **exceptions**:

exception 1: what do we do if the OB or BB does not receive a response from the CB if its API is down?

exception 2: what happens with PO's in CB.PO_OUT or CB.ACK_OUT when no request is received from OB or BB?

Task B: create and set up a **repository** for your team. Try to explore some more advanced features during setup: who has push access, who can create pull requests? Are collaborator reviews required before a pull request can be merged? How about branches? Etc..

Task C: Set up **project management**: board (Trello, Kanban...), define and assign individual tasks (building the database, writing the report, designing the API...)



Day 1 – afternoon: presentation, simulation and design

The afternoon starts with a **short presentation** by each team to share the work done this morning: analysis of general messaging scheme use cases, project management setup... After that, you'll test the CB API, start building the database and design your application. First decide on who will do what (and update your project management board).

Task D: Simulate a complete payment cycle using the sample CB on <https://stevenop.be/pingfin/api/v2/>, not with code, but just with [Postman](#), Excel and some other tools. First become familiar with the basics of the API:

- **API 1:** you should have received two secret keys for two banks; **generate a token** for each bank (will be valid for 4h)
- **API 2:** try to get a list of active banks (list should change as more and more banks register)
- **API 3:** post your member info to your bank

Now you're ready for a full-scale **simulation** of a payment cycle with spreadsheets and Postman.

- first create **two spreadsheets**, one for the OB and one for the BB, with all relevant tables on separate tabs (e.g. OB: at least ACCOUNTS, PO_NEW, PO_OUT, ACK_IN, TRANSACTIONS and LOG)
- find a way to **generate PO's** (manually? Excel formula's? some code?); also include some errors (negative amount, non-existing BB etc...)
- now **send the PO's** to the CB
- next check CB.ACK_OUT for **failed PO's**; process them in your OB spreadsheet
- fetch PO's from the CB into BB
- process then in your OB's spreadsheet
- etc...

Take screenshots of each step!



Day 1 – afternoon: presentation, simulation and design

Task E: design your future application. Some questions to consider:

- how will you generate random PO's?
- API design – what endpoints will you need?
- how will you document the API?
- what are the building blocks of the back-end?
- which external libraries and tools will you use?
- how will you build the front-end (mock-ups, languages, libraries, CSS framework...)?
- ...

→ create a **visual model** (cfr. flowchart) that shows the basic building blocks and their interactions

Task F: start building the application, starting with the database.



Day 1 – deliverables

- ✓ Github repository with read access for the coach
- ✓ Word document (published on the repo) with:
 - A. title page (year, title, institution, team members...)
 - B. metadata (workshop context, team member roles...)
 - C. visual representation of the basic messaging scheme use cases
 - D. strategies to recover from exceptions
 - E. software design: global structure of the application, tools/libraries/packages/frameworks used...
- ✓ Trello/Kanban/... project management board setup; be sure to log team members tasks, progress and activities
- ✓ Short presentation for the afternoon with (not necessarily in this order):
 - A. team members and their roles
 - B. chosen bank (CB or OB/BB)
 - C. repo and project management setup
 - D. visual representation of the basic messaging use cases
 - E. other...
- ✓ SQL file with the database scheme + data
- ✓ Screenshots of Excell/Postman simulation



Day 2 – implementation

Day 2 start with **teams meeting**, where each team plans the work for this day and assigns tasks to individual members (cfr. project management board). The rest of the day, you will focus on **API design and implementation and building the GUI**.

Suggestions for **morning** activities:

1. API endpoints design (url, POST or GET, headers, expected get parameters and body...)
2. write code to generate random PO's

Suggestions for **afternoon** activities:

3. start with the API implementation
4. create an API documentation
5. start with API testing
6. start **building the GUI** (start with creating mockups, e.g. in Pencil, Balsamiq, LucidCharts or similar)

Notes:

- *You probably won't finish the API today. It would be great if you did five endpoints, with at least one with PO's in the body.*
- *For the API documentation, you could use plain HTML, but try to find more professional solutions*
- *For API testing, until the GUI is ready, we would recommend starting with [Postman](#). You all should be (sort of) familiar with this tool.*



Day 2 – deliverables

- ✓ add API documentation to report document (not just a link)
- ✓ add GUI mockups to report document



Day 3 – validation, testing & deployment

Day 3 again start with teams meeting, where each team plans the work for this day and assigns tasks to individual members. The rest of the day, you will focus on writing validation-logic and testing your **API with other banks**.

Suggestions for **morning** activities:

1. finish your **API implementation and testing**
2. create a list of **error codes** you'll use; this will give a good idea of what you'll validate and check
3. implement **validation** and **exception handling** in your code

Suggestions for **afternoon** activities:

4. try to complete a **full test run**, i.e. generate and test PO's with payments to a bank from another group through a clearing bank
5. finish **building the GUI**
6. deploy your Docker-containers on **server**



Day 3 – deliverables

- ✓ add test run details and results to report document



Day 4 – final presentation

Day 4 is fully committed to the final presentations (about 30 mins per team). Focus equally on product *and* process.

Some topics for **product**:

- scheme of your application design
- noteworthy code fragments
- what has been realized and what not
- performed tests / demo (tip: it is ok to show a pre-made video instead of a live demo)
- ...

Some topics for **process**:

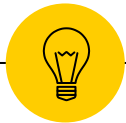
- project management: tools, roles, tasks...
- communication/collaboration: repository setup, communication tools, difficulties...
- retrospective: what went well, what went bad, what have you learned, what would you do differently a next time...
- suggestions to improve the workshop for future editions
- ...



Day 4 – deliverables

- ✓ add final reflection to report document
- ✓ final presentation in Powerpoint format
- ✓ upload all code and deliverables in a single zip file on Toledo (**one upload** per team)

Day by day (network team)





Day 1 – setup & preparation

Start the day with a team meeting, where the network team aligns on the goals for the week, divides responsibilities, and plans the setup work. The focus of this day is on building a solid foundation: hardware setup, network configuration, and preparing the server environment.

Suggestions for **morning** activities:

1. collect all required hardware (server, switch, cables)
2. set up the physical network infrastructure
3. ensure basic connectivity between devices
4. install and configure the server OS

Suggestions for **afternoon** activities:

1. install and configure Docker on the server
2. define deployment requirements (ports, naming conventions, image format, etc.)
3. communicate clearly with bank teams about expectations
4. perform initial connectivity and environment tests



Day 1 – deliverables

- ✓ working local network (wired)
- ✓ operational server with Docker installed
- ✓ clear deployment guidelines for bank teams



Day 2 – integration & deployment

Start the day with a short team meeting to align on priorities and define how dummy services will be used to validate the infrastructure. Since the real bank applications are not yet available, the focus is on testing the deployment pipeline and inter-container communication using simple placeholder APIs.

Suggestions for **morning** activities:

1. define a minimal dummy API specification (e.g. simple GET endpoint returning JSON)
2. create and deploy test containers (own or provided by teams)
3. verify that containers can be built, shared, and deployed correctly
4. test basic network connectivity between containers

Suggestions for **afternoon** activities:

1. implement a dummy API that:
 - retrieves data (e.g. from a small database or static dataset)
 - exposes it via a simple JSON endpoint
2. implement a second API/container that:
 - calls another container on the same server
 - verifies inter-container communication
3. test communication via IP, hostname, or Docker networking
4. troubleshoot and document common issues



Day 2 – deliverables

- ✓ working deployment flow using Docker images
- ✓ successfully running dummy containers on the server
- ✓ verified inter-container communication
- ✓ clear guidelines for bank teams (how to structure and expose their APIs)



Day 3 – stabilization & optimization

Start the day with a planning session to prepare for integrating the real applications. The focus shifts from testing with dummy services to deploying and stabilizing the actual bank applications.

Suggestions for **morning** activities:

1. collect real Docker images from bank teams
2. deploy the actual bank applications on the server
3. verify that each service is reachable

Suggestions for **afternoon** activities:

1. test real API interactions between teams
2. troubleshoot integration issues (networking, ports, configs)
3. optimize routing (e.g. consistent ports or reverse proxy if applicable)
4. finalize documentation and assist teams with final fixes



Day 3 – deliverables

- ✓ real bank applications deployed and running
- ✓ working API communication between teams
- ✓ stable and consistent deployment setup
- ✓ finalized documentation



Day 4 – final presentation

Day 4 is fully committed to the final presentations (about 30 mins per team). Focus equally on product *and* process.

Some topics for **product**:

- overview of your network architecture (server, switch, topology, Docker setup)
- explanation of your deployment setup (how containers are built, delivered, and deployed)
- how you ensured connectivity between all teams
- noteworthy technical implementations (e.g. Docker networking, routing, monitoring, ...)
- challenges encountered (e.g. networking issues, container conflicts, configuration problems) and how you solved them
- what has been realized and what not
- performed tests / demo (e.g. showing inter-container communication, deployed services, monitoring, ...)...

Some topics for **process**:

- project management: task division, planning, coordination with multiple teams
- communication/collaboration: how you interacted with bank teams, collected requirements, handled dependencies
- documentation: how you structured and shared technical info (ports, endpoints, deployment steps, ...)
- difficulties: working with hardware, networking constraints, dependency on other teams, timing issues
- retrospective: what went well, what went bad, what have you learned, what would you do differently next time
- suggestions to improve the workshop (especially regarding coordination between dev & network teams)...



Day 4 – deliverables

- ✓ deployment & setup documentation
- ✓ Docker configuration (compose or commands)
- ✓ network/architecture diagram
- ✓ connectivity & testing report
- ✓ overview of deployed services
- ✓ issues encountered + solutions